

Splay strategies for splay trees

The splay tree uses three types of rotations to *splay* a key into the root: zig, zig-zig and zig-zag. A question that may come to mind is how the splay tree behaviour would change if only zig rotation can repeatedly be applied to splay a key.

The task

Implement the standard splay tree and also a splay tree with modified splay strategy, which uses d zig rotations to move a key in depth d into the root. Use a sequence of Insert and Delete operations to test both types of splay trees and compare their final heights.

Assume that

- Insert(K) is never called when key K is already contained in the tree,
- Delete(K) is never called when key K is not contained in the tree.

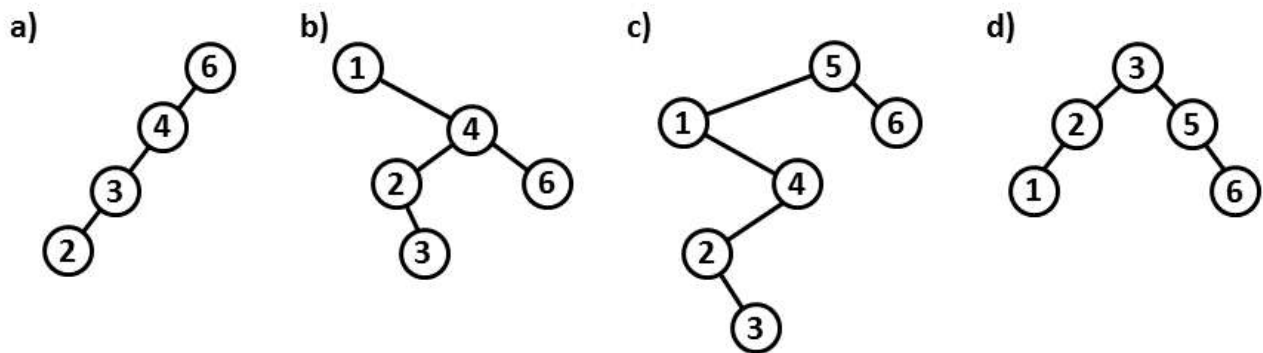


Image 1. States of the standard splay tree after a) inserting keys 2,3,4,6, b) inserting key 1, c) inserting key 5, d) deleting key 4.

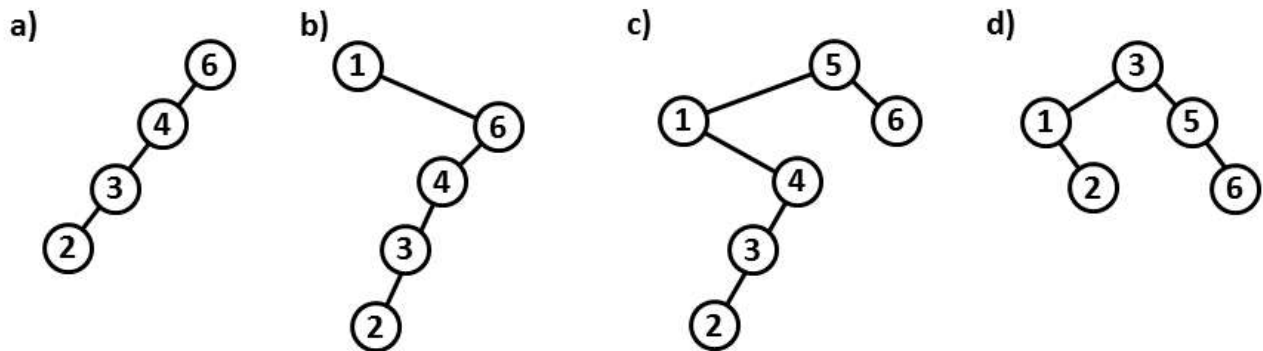


Image 2. States of the modified splay tree (that uses only zig rotations) after a) inserting keys 2,3,4,6, b) inserting key 1, c) inserting key 5, d) deleting key 4.

Input

The input consists of two lines. The first line contains an integer N representing the number of operations to be performed. The second line contains N non-zero integers separated by spaces. Each positive integer K represents Insert(K) operation, while each negative integer represents Delete($|K|$) operation. The integers order corresponds to the order in which the operations are performed.

It holds that $1 \leq N \leq 1.2 \times 10^6$. Each key to be inserted/deleted is not greater than 1.2×10^7 .

Output

The output is one line containing integers H_1 , H_2 separated by a space. Integer H_1 is the height of the splay tree with the standard splay strategy after performing all N operations. Similarly, H_2 is the height of the modified splay tree (which uses only zig operations to splay a key into the root) after performing all the operations.

Example 1

Input

```
7
2 3 4 6 1 5 -4
```

Output

```
2 2
```

Solution of Example 1 is visualized in Image 1 and Image 2.

Example 2

Input

```
11
3 4 5 6 8 7 9 2 1 -6 -8
```

Output

```
4 5
```

Example 3

Input

```
19
124 26 128 186 101 97 92 142 54 -186 13 45 120 175 176 177 41 179 114
```

Output

```
6 8
```

Public data

The public data set is intended for easier debugging and approximate program correctness checking. The public data set is stored also in the upload system and each time a student submits a solution it is run on the public dataset and the program output to stdout and stderr is available to him/her.

[Link to public data set](#)